How many rows and columns did pandas load?

- Pandas loaded 144,724 rows and 15 columns with the default settings.

How many rows does each version load? Did pandas warn you?

- The default load gave 144,724 rows. Loading with quoting=3 gave 145,431 rows. That is a difference of 707 rows. Pandas did not show any warning or error, so the missing rows were lost silently.

What does that tell you about checking your data?

- Pandas did not complain, but the default settings dropped 707 reviews. Stray quote marks in the text made pandas read some reviews as part of the next row, so rows got merged together. Code that runs without errors can still give you wrong data. You have to check the row count against what you expect, and compare different loading settings, before you trust any results built on top of it.

What looks weird?

- Several reviews contain HTML leftovers. I saw
 line break tags inside sentences, " codes where quote marks should be, and \\" escape characters in one review. There are also typos and uneven spacing.

Where do you think it came from?

- The reviews were written on a website that stores text as HTML. When the data was exported, the HTML formatting was saved along with the text instead of being converted to plain characters.

How are the ratings distributed? Why does that matter for accuracy?

- Most reviews are positive. About 55% are 5 stars and about 70% are 4 or 5 stars. Only about 5% are 2 stars and 8% are 3 stars. This matters because a model that said “positive” for every review would still be right about 70% of the time without understanding anything. A high accuracy score on this data could just reflect the imbalance, not a good model.

What do you notice about the most-reviewed “games”?

- Many of the most reviewed items are not games at all. They are gift cards, points cards, and subscriptions. About 22% of all reviews are for these five products. Reviews of a gift card talk about delivery, redemption codes, or customer service, not gameplay. So the dataset is not purely game reviews, and a sentiment model tested on it is partly being tested on reviews about codes and billing.

How many reviews had line breaks before cleaning, and after?

- 25,699 reviews had
 tags before cleaning. After cleaning, 0 did.

What still looks weird in the five reviews? Would it change whether a model reads a review as positive or negative?

- The
 tags are gone, but other leftovers remain. Review 1 still has "AI" and review 4 still has "Trouble Communicating With Gateway". Review 5 still has \\"seems\\" and \\"free\\". There are also typos like “payed” and “pena lized”, and uneven double spacing.
- These leftovers probably would not flip a review from positive to negative, because the sentiment comes from words like “bugs”, “screwed”, and “crapware”. But they could make a model less certain. The quote codes break up the words that reviewers put in quotes for sarcasm, like “free” and “seems”, and a model may not read that sarcasm correctly. So I would say cleaning helped, but it was only partial.

Why take the same number of each star rating instead of 500 random reviews?

- In a random 500, about 277 would be 5 stars, but only about 27 would be 2 stars and 40 would be 3 stars. Those middle ratings are the hardest for a model, and you would barely have any to test on. Taking 100 of each star rating gives every rating enough examples to judge fairly.

What are the keys in the result, and what does the score mean?

- The keys are label and score. The label is the model’s predicted sentiment, here NEGATIVE. The score is the model’s confidence in that label, a number between 0 and 1. A score of 0.9998 means the model is almost completely sure the review is negative.

Does the model agree with the customer’s star rating?

- Yes for this review. The customer gave 1 star and the model said NEGATIVE with very high confidence. This is only one review, so it does not tell me how the model does overall

About how long did 500 reviews take? At that rate, how long would all 145,000 take?

- 500 reviews took 36.6 seconds. At the same rate, all 145,000 reviews would take about 3 hours. That is too slow to run casually, which is why we test on a sample first.

What does truncation=True protect against?

- The model can only read a limited amount of text at once, about 512 tokens for this model as far as I know. Some reviews run over 5,000 words. Without truncation, those reviews would be longer than the model can accept and the call would likely fail, which could stop the whole batch. With truncation, the model cuts the text off at its limit and reads only the beginning. The trade-off is that for very long reviews, the model never sees the ending.

Which star rating does the model get wrong most often?

- The 4 star reviews. The model labeled 23 of 100 as NEGATIVE, so it was wrong 23% of the time. The 5 star reviews were next at 17%. It was wrong on only 4 to 5% of the 1 and

2 star reviews, so the model is much better at spotting unhappy customers than happy ones.

The model only knows POSITIVE and NEGATIVE. What did it do with the 3 star reviews?

- It had to pick a side. It called 66 of them NEGATIVE and 34 POSITIVE, so it leaned negative. A 3 star review is usually mixed, such as praise for one thing and complaints about another, and the model has no neutral option for that.

Review 1 (SimCity)

- The review has real praise (“awesome”, “graphics are great”) and ends with “overall it’s a good game”. But the reason for 1 star is the server problem that loses your progress. The model followed the many positive words. A human would weigh the “BUT” more heavily. I would call it a mixed review where the star rating is harsher than the final verdict, so both sides share the blame.

Review 2 (“Enjoys game”)

- The text is clearly positive and the model read it correctly. The 1 star was probably a mis-click or a rating given for something else, like delivery. This is the customer’s rating being misleading, not a model failure.

Review 3 ($7 wasted)

- This one is the model’s fault. The customer is clearly unhappy: “wasted $7”, “cant even get to the game part”, “tried a lot”. The model got stuck on “im sure its a good game”. The low confidence of 0.73 shows it was less sure here than on the others.

Review 4 (handheld port)

- The customer says it does not work right and was “not happy”, but the review ends with “fun game love twisted metal games”. The 1 star is about the technical problem, while the text is about liking the game. The model went with the positive ending. Both sides share the blame again.

Review 5 (daughter’s game)

- This is the model’s fault. The review is polite and uses soft wording like “unfortunately”, and the early words like “likes” and “very simple” are not strongly negative. A human sees that the game looked dated and did not hold the kids’ attention, which is negative. The model was fooled by the polite tone.

How often did it get the exact star rating? How often within one star?

- The model matched the exact star rating 57.4% of the time. It was within one star 91.2% of the time. So it usually lands close even when it misses.

Which model would you use for a sentiment dashboard for a game publisher, and why?

- I would use the star model. The first model only says POSITIVE or NEGATIVE, so it cannot separate a mildly disappointed 2 star customer from an angry 1 star one. It also has to force mixed 3 star reviews into one side, and in Part 3 it called 66 of them negative. The star model gives a 1 to 5 score, which a publisher can track over time, compare across games, and use to see how strongly customers feel. Since it is within one star 91% of the time, the trends it shows are reliable even if single predictions are sometimes off.

What customers complain about most?

- The most common complaints are download or install problems and bugs or crashes, with 5 reviews each. Adding account or login problems, about 70% of the one-star complaints are technical issues that stop people from using the product. Only 3 were about price and 3 about gameplay. For a publisher, that suggests most one-star reviews are about the product not working, not about the game being bad.

Three reviews with topics. Does the choice make sense?

- Mostly yes. Review 2 is a clear install problem and review 3 is about slow pacing, though it is sarcastic. Review 1 is a judgment call. Price fits the “$30 extra” complaint, but the main complaint is the interface, which none of my labels cover. The model can only choose from the labels I give it.

What happened to the top labels and their scores?

- For 4 of the 5 reviews the top label stayed the same, but its score dropped every time. Price went from 0.66 to 0.59, download or install from 0.41 to 0.30, gameplay from 0.36 to 0.32, and download or install on the last review from 0.79 to 0.45. Review 4 changed its top label from bugs or crashes (0.53) to technical problems (0.70).

The scores add up to 1. How does that explain what you saw?

- All the labels share one total of 1. Before, five labels split it. After, seven labels split it, so the two new labels took part of the total from the old ones. That is why the old top scores fell even though the reviews did not change. In review 4, technical problems was close enough in meaning to bugs or crashes that it took the top spot.

What does this tell you about choosing labels for a real product?

- A label’s score depends on what else is in the list, not only on the review. Labels that overlap, like technical problems and bugs or crashes, split the score and can change the top label. Even an unrelated label can take a small share. So a real label list should be short and clearly different from each other, and tested on real reviews before anyone trusts the counts. A broad label like technical problems also tells a publisher less than a specific one like bugs or crashes.